

出力・可視化の方針

最終更新: 2026-08-12 (HBN240018)

最終的に作りたい出力の構成と、そのために各計算段階で何を算出・蓄積しておく必要があるかをまとめた文書。可視化そのものの実装は後回しでよいが、これから書くコードは常にここに必要な情報を落とせる形にしておくこと。

想定している出力構成

#	出力	用途	データ源	状況
①	音線の可視化	反射経路を線で確認する	音線法 (③) の軌跡	完了 (<code>view_rays.py --mode rays</code>)
②	音粒子の可視化 (動画)	離散化時間ごとに粒子が飛ぶ様子。音の広がりを見る	音線法 (③) の軌跡	完了 (<code>--mode particles</code> 、GIF 出力あり)
③	インパルス応答	—	⑥ impulse.py	完了 (CSV + <code>impulse_response.png</code>)
④	各種音響指標	T30, EDT, C50, D50 など	③のIRから	完了 (<code>reverberation.png</code> / <code>clarity.png</code>)
⑤	伝搬方向確認 (図)	受音点に人の頭の絵を置き、どの方向成分が多いか	⑤ バックトレースの出力	完了 (<code>direction.png</code>)
⑥	モード分布	受音点における	③のIRから／別途	完了 (<code>modes.png</code>)
⑥-2	モードの積み上げと吸音の効果	経路差から強め合う周波数を数え、吸音の有無で比べる	⑤ バックトレースの出力	完了 (<code>mode_buildup.png</code>)

(番号の①～⑥は出力の識別番号。パイプラインの①～⑦とは別物)

モデルビューア (先に作成済み・2 種類)

上記 6 種の前に、読み込んだモデルを確認するビューアを作っている。表示内容は同じで、実装方式が違う 2 本を並存させている。

```
cd geosim
python view_model.py      ../test2.dxf --absorption ../absorption.csv # HTML + WebGL
python view_model_gui.py  ../test2.dxf --absorption ../absorption.csv # PyVista ウィンドウ
```

- 三角形要素 (辺を描くので分割が見える) / 法線ベクトル (矢印) / レイヤ別の色分け
- 法線の裏側を赤で塗るので、向きの誤りが一目で分かる

- 回転／拡大縮小／平行移動／視点プリセット
- 音源・受音点、読み込み結果のサマリ

	view_model.py	view_model_gui.py
方式	自己完結 HTML + WebGL を書き出してブラウザで開く	PyVista (VTK) のネイティブウィンドウ
依存	なし (標準ライブラリ + numpy)	pyvista + vtk
強み	相手に環境構築を求めずに共有できる	Python から直接操作できる
発展先	Web ベース GUI	①②の重ね描き、デスクトップ GUI

HTML 版を先に作ったのは、当時この環境に matplotlib / pyvista が入っておらず (tkinter のみ)、pip install を前提にしなかったため。2026-08-14 に Python 3.10.11 の venv を切って pyvista を導入し、ネイティブ版を追加した。HTML 版は捨てていない。共有用途では今も HTML 版のほうが便利のため。

したがって GUI (G-7) は Web ベース / デスクトップのどちらに倒しても土台がある状態。

音線・音粒子ビューア (①② 実装済み)

geosim/view_rays.py。上のモデルビューア (PyVista 版) に重ねて表示する。

```
cd geosim
# ① 音線 (受音した経路の初期反射だけ、到来時刻で色分け)
python view_rays.py ../test.dxf ../結果/test_raylog.npz --received-only --max-reflection 3 --color time
# ② 音粒子 (スペース 再生/停止、← → コマ送り、スライダで時刻指定)
python view_rays.py ../test.dxf ../結果/test_raylog.npz --mode particles
# ② を GIF に
python view_rays.py ../test.dxf ../結果/test_raylog.npz --mode particles --movie 広がり.gif
```

やってみて分かったこと

- 音線は本数を絞らないと読めない。小さい室で 250 本 × 13 反射を描くと

線が重なって何も分からなかった。既定を 80 本にし、

--max-reflection で折れ線を途中で打ち切れるようにした。

「その回数以下の音線だけ残す」ではないのが要点で、閉じた室では全音線が上限まで反射するため絞り込みでは何も残らない

- 受音した経路だけを見るのがいちばん分かりやすい。 --received-only
- 音粒子は全音線ぶん出したほうがよい。こちらは点なので重なっても密度として読める。

むしろ本数が多いほど「広がり」が見える

- 動画は追加の依存を増やさず Pillow で GIF にした

(60 コマ 960×720 で 3.3 MB。長くするなら間引きかフレーム数の調整が要る)

- 壁の透過はレイヤごとに換えられるようにした (ユーザー要望、2026-08-14)。

左の縦スライダ、`Tab` で対象切替、`m` でモデル表示 ON/OFF。

起動時指定は `--opacity` と `--layer-opacity "1=0.6,2=0.05"`

- 再生は VTK のタイマーでは動かなかった。 `add_timer_event` が発火せず、

「コマ送りは効くのに再生されない」状態だった。

`show(interactive_update=True)` + 自前ループに変更して解決

GUI の全体設計 (G-7) は後回しにしている (ユーザー判断)。ここは「見るための最小限」に留めてあり、将来 GUI に組み込むときは `RayLog` と `ParticleAnimation` をそのまま使える形にしてある。

重要な前提：最終的な計算 (到来時刻・エネルギー) は音線法ではなく虚音源バクトレースで行うが、音線法の段階のデータも捨てずに蓄積する。出力①②は音線法ベースのデータを使う。

出力①②のためのデータ — 音線軌跡

対応コード: `geosim/ray_recorder.py`、`loop_reflectionmesh.loop(..., recorder=...)`

なぜ本線と分けたか

音線追跡ループの「本線」の出力は**受音した経路の反射面 ID 列** (元コード `traceff`) だけ。一方、可視化で見たいのは「音がどう広がるか」なので、**受音しなかった音線も必要**。目的が違うので同じ配列に混ぜず、`RayRecorder` を副チャンネルとして渡す設計にした。`recorder=None` なら一切記録しないので、本番計算の速度・メモリには影響しない。

記録している内容 (`RayTrajectory`)

項目	形状	内容	使い道
<code>ray_index</code>	int	間引き前の元の音線番号	再現性・デバッグ
<code>direction</code>	(3,)	音源から出たときの初期方向	出射方向で色分けする等
<code>nodes</code>	(m+1, 3)	節点座標 (先頭が音源、以降が各反射点)	①折れ線として描画
<code>distances</code>	(m+1,)	音源からの累積距離 [m]	②時刻の算出
<code>mesh_ids</code>	(m,)	各セグメント終端で反射した面 ID	壁ごとの色分け
<code>energies</code>	(m+1, 6)	各節点でのバンド別エネルギー	線・粒子の減衰表現
<code>receive_steps</code>	(r,)	受音したセグメント番号 k	受音経路のハイライト
<code>termination</code>	str	'no_hit' / 'nref'	打ち切り理由の把握

- 時刻は `traj.times(c) = distances / c` で得る。
- 音粒子アニメーションは `traj.position_at(t, c)` を全軌跡について毎フレーム呼べばよい

(セグメント間を線形補間して位置を返す。未出発／消滅後は `None`)。

- 保存は `recorder.save_npz(path)`。可変長なので**連結＋オフセット方式**で持つ

(`nodes[node_offsets[i]:node_offsets[i+1]]` で *i* 本目が取れる)。

間引きの方針

100 万本飛ばしても全部は描かないので間引きが要る。「**総数比 1/100**」ではなく「**絶対本数の上限＋等間隔ストライド**」を既定にした (`max_rays`、既定 2000 本)。

理由:

- 総数比だと描画本数が安定しない**。1/100 は 100 万本なら 1 万本 (重くて描画が破綻する)、
1000 本なら 10 本 (少なすぎて広がりが見えない)。可視化の品質は「何本描くか」で決まるので、そこを直接指定するほうが素直。
- 絶対本数なら描画性能が音線数に依存しない**。 `stride = total // max_rays` で自動追従する。
- 音線数を変えて計算しても、**見た目の密度が変わらないので比較しやすい**。

対話的に「もっと細かく／粗く」を切り替えたい場面は出てくるので、`max_rays` は GUI 側から渡せるパラメータにしておく想定。

注意：Fibonacci 螺旋は添字 *i* に対して高さ *z* が単調増加、方位角が黄金角ずつ回る配置なので、等間隔ストライドで抜いても *z* は一様なまま (= 等立体角は保たれる)。ただしストライド幅と黄金角の関係次第では方位角に縞模様が出る可能性がある。描画してみて偏って見えたら、固定シードのランダム抽出に切り替える。

出力⑤ のためのデータ — 伝搬方向

対応コード: `loop_noredundancy.py` (バックトレース)

元コード 1080 行の出力にすでに含まれている。

```

ktmp, rtime, -vtgt(1:3), enertmp(1:6)
↑      ↑      ↑      ↑
反射回数 到来時刻 到来方向   バンド別エネルギー

```

到来方向ベクトル (3～5 列目) がそのまま出力⑤ の素材になる。受音点を原点として、この方向ごとにエネルギーを積み上げれば方向分布が描ける。人の頭の絵を置くなら、頭部座標系 (正面・上方向) を別途

入力として持たせる必要がある。

- 時間で区切った方向分布（初期反射だけ／後部残響だけ）も見たくなる見込みなので、

到来時刻とセットで保持しておくこと。

- バンド別に分けられるようにしておくこと（低音と高音で方向性が違う）。

出力③④⑥のためのデータ

- ③ インパルス応答 … `impulse.py` の出力そのもの。CSV で時間ベクトル付き
- ④ 各種音響指標 … ③の IR から算出。Schroeder 積分 → T30 / EDT、

初期/後期のエネルギー比 → C50 / D50。バンド別に出せるようにしておく

- ⑥ モード分布 … IR の FFT。低域の固有周波数分布を見る用途。

書籍 2.1「波動音響理論」（固有振動・直方体音場・固有周波数分布）が理論的な下敷き

- ⑥-2 モードの積み上げと吸音の効果 … ⑤ のパルス列を位相込みで足す。

到来時刻の差＝経路差なので、経路差が波長の整数倍になる周波数で強め合う。

減衰を考えなければ $|H(f)|$ は「その周波数で同位相に重なった経路の本数」そのもの。

重みを $1 \rightarrow 1/d$ （完全反射）→ $\sqrt{e}/d$ （設計の吸音）と変えて 3 本描き、

後ろ 2 本の差を吸音の効果として塗る。

パルスのエネルギーには面の吸音しか入っていない（距離減衰と空気吸収は

`impulse.py` 側で掛ける約束）ので、3 本をきれいに分離できる。

計算は「時刻をヒストグラムにして FFT」なので $O(n \log n)$ 。処理は増えない

これから実装するときのチェックリスト

新しい計算段階を書くときは、以下を満たしているか確認する。

- ☐ 時刻を保持しているか。②の動画も⑤の時間区切りも時刻がないと作れない
 - ☐ バンド別のまま保持しているか。合算するのは最後の描画段階でよい
 - ☐ 受音しなかったデータも要るか考えたか。①②は要る、③④⑤⑥は不要
 - ☐ 間引き・保存が本線の速度に影響しない形か（optional な副チャンネルにする）
 - ☐ GUI からパラメータを渡せる形か（ハードコードしない）
-

未決事項

- GUI のフレームワーク（Web ベースか、PyQt などのデスクトップか）

※どちらでも土台がある。Web なら `view_model.py`、デスクトップなら `view_model_gui.py`。
デスクトップに倒す場合、ウィンドウに他のウィジェット（スライダ・グラフ）を同居させるなら
`pyvistaqt` + `PySide6` を追加導入することになる（未導入）

- 音粒子アニメーションのレンダリング方法（Python 側で動画書き出しか、データを渡してブラウザで描くか）
- 出力ファイルのフォーマット統一（現状 npz と CSV が混在）
- 頭部座標系（正面・上方向）の入力方法 — 出力⑤に必要